

1 1.D - Outils

Dans cette section, nous présentons trois outils essentiels pour suivre ce module :

- le **terminal** qui permet d'exécuter des commandes et naviguer dans une arborescence de fichier,
- **Wireshark** qui est un outil de capture et d'analyse des trames qui passent dans une carte réseau,
- et **Filius** qui est un outil pédagogique de simulation de réseaux.

1.1 Utilisation du Terminal

1.1.1 Présentation

Un **terminal** est une fenêtre dans laquelle on peut taper des commandes qui sont exécutées par le système d'exploitation. On retrouve cette application sous d'autres termes : *console*, *shell*, *invite de commande*, etc.

Pour exécuter une commande ou lancer un programme déjà installé, il suffit de taper son nom dans le terminal.

Par exemple, pour lancer un navigateur :

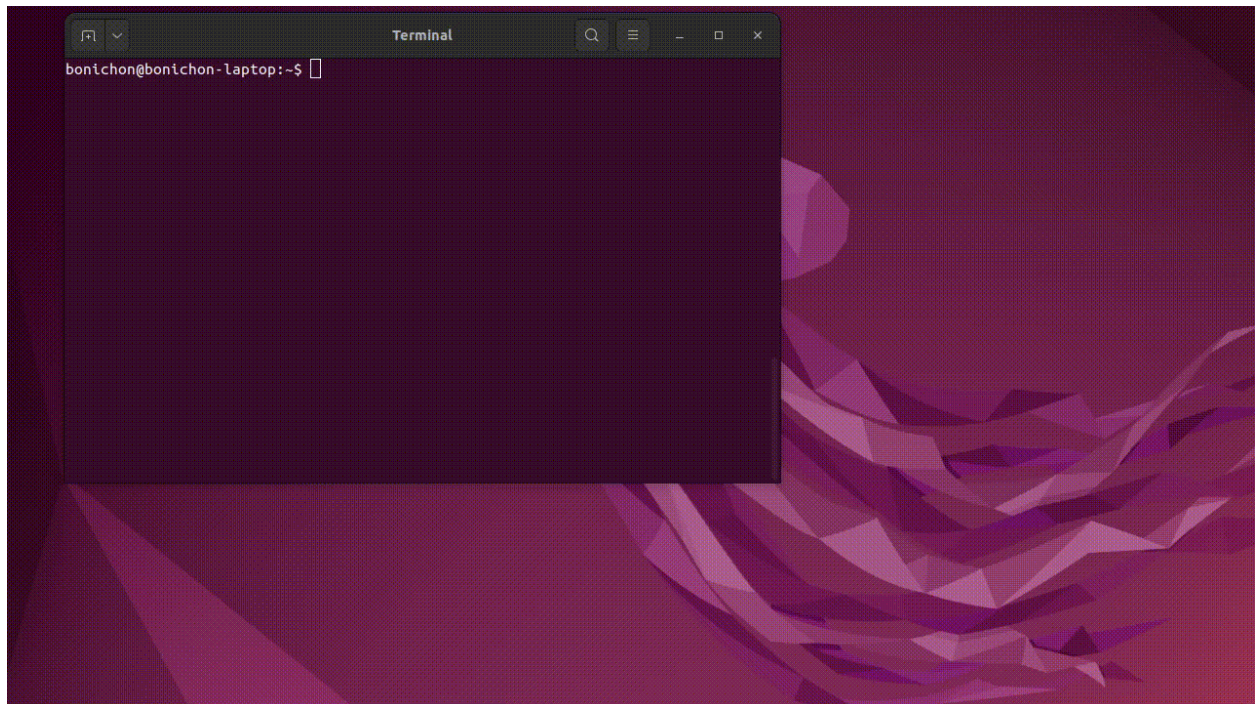


Figure 1: lancement d'un navigateur

Deuxième exemple, la commande `date` me permet d'afficher la date :

Le terminal exécute en boucle la séquence d'actions suivante :

- affichage de l'invite de commande (*prompt*) signalant qu'il est en attente d'une nouvelle commande de la part de l'utilisateur ;
- aide à la saisie de la nouvelle commande, en fournissant à l'utilisateur des fonctions d'édition de texte en mode ligne : accès à l'historique des commandes précédentes, copie et remplacement rapides, complétion de noms de fichiers, etc. ;
- analyse et vérification syntaxique de la commande tapée par l'utilisateur, et affichage d'un message d'erreur en cas de problème ;

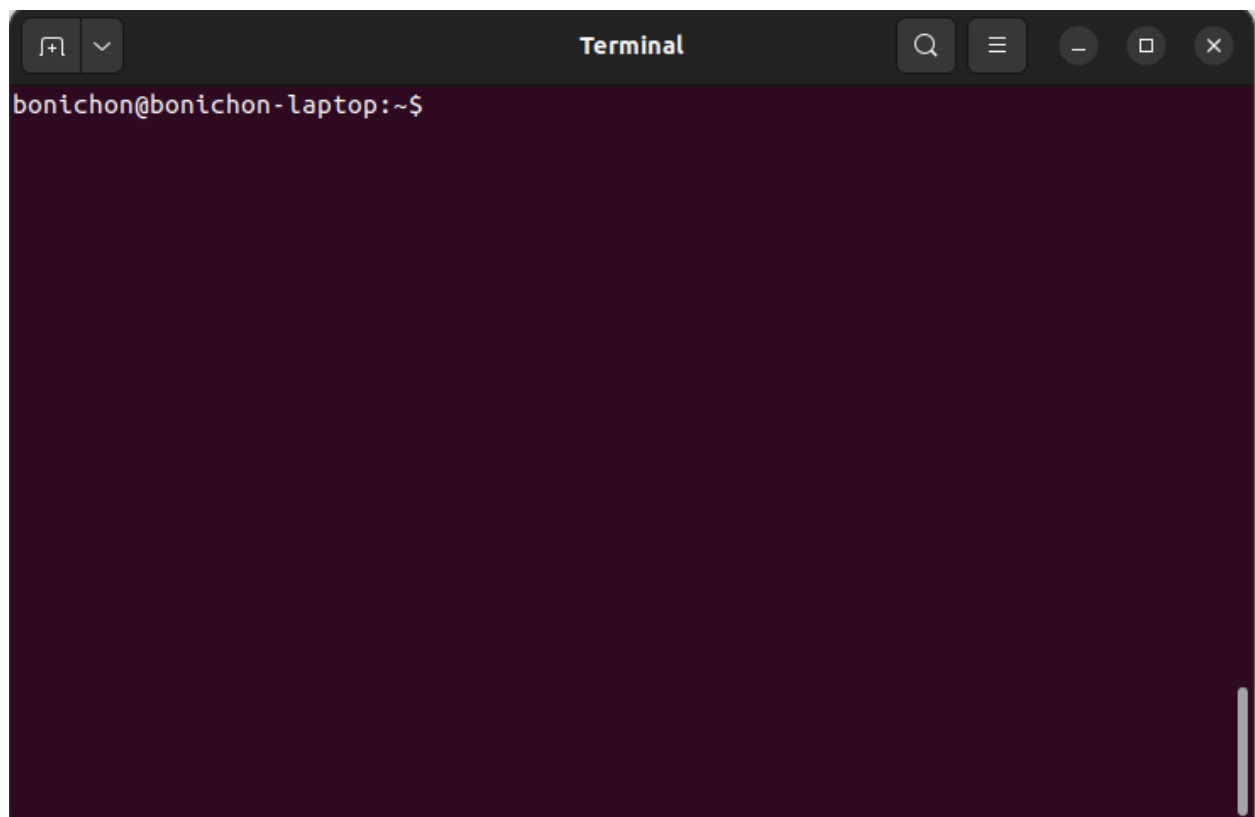


Figure 2: lancement de la commande date

- si la commande est conforme, un appel au système d'exploitation est effectué pour réaliser les opérations demandées : lancement d'un nouveau processus, etc. ;
- attente de la fin de la commande en cours.

Plus d'info : http://fr.wikipedia.org/wiki/Bourne-Again_shell

1.1.2 Le terminal que sous Linux ?

Nous détaillons ici l'utilisation d'un terminal sous Linux, mais il existe des terminaux sous Mac et Windows.

Sous Mac cela fonctionne exactement de la même manière :

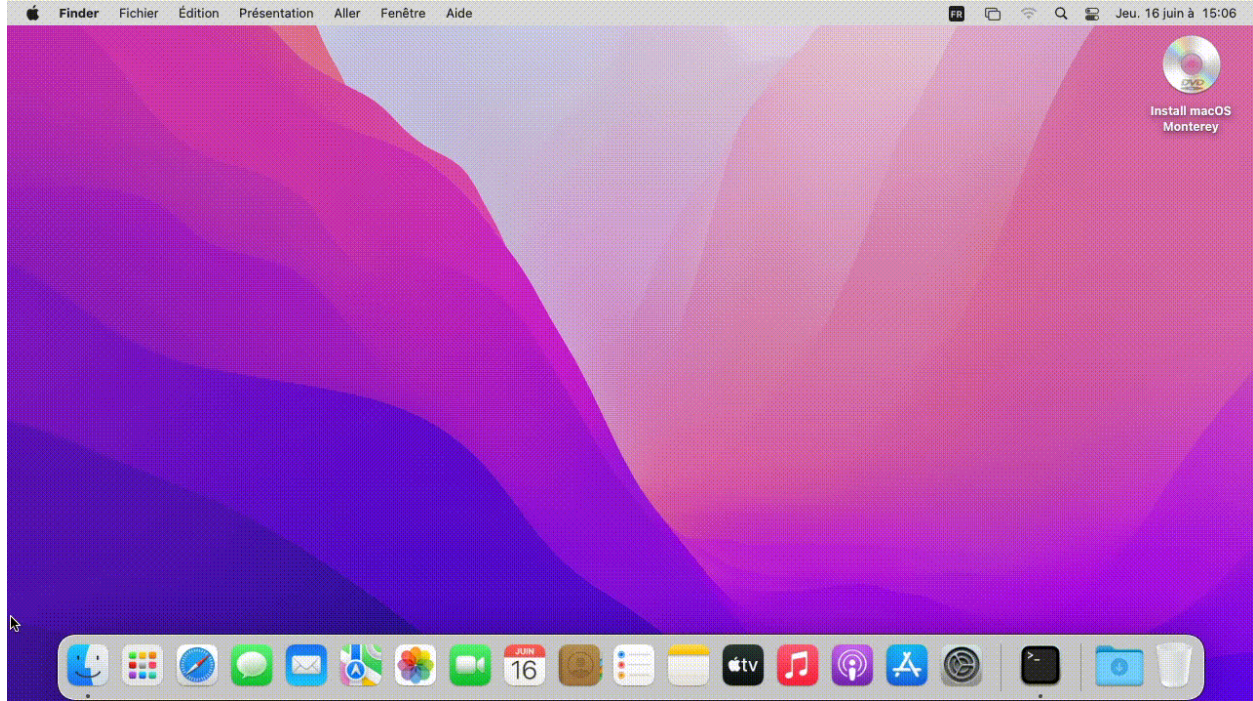


Figure 3: Lancement d'un terminal sous mac

Sous windows, c'est un petit peu plus subtile. Il y a un terminal historique qui s'appelle l'invite de commande (ou `cmd.exe`). Sur les dernières versions, il existe un autre terminal appelé **powershell** qui est un peu plus puissant et dont l'utilisation se rapproche plus de celle d'un terminal sous Linux.

Le principe reste le même, la syntaxe est néanmoins un petit peu différente.

1.1.3 Manipulation du système de gestion de fichiers

Mais revenons à l'utilisation d'un terminal. Commençons par les commandes qui permettent de manipuler votre système de gestion de fichiers, un peu comme vous le feriez avec un explorateur de fichiers. Avant cela, commençons par quelques notions.

1.1.3.1 Système de Gestion de Fichiers Unix

Le système de gestion de fichiers (SGF) d'Unix procure à l'utilisateur un moyen efficace pour conserver et manipuler aisément des informations. En outre, il offre un système de sécurité, notamment concernant les droits d'accès aux fichiers. Il existe trois principaux types de fichiers :

- les fichiers de données (ordinary files),
- les répertoires (directories) ou dossiers,

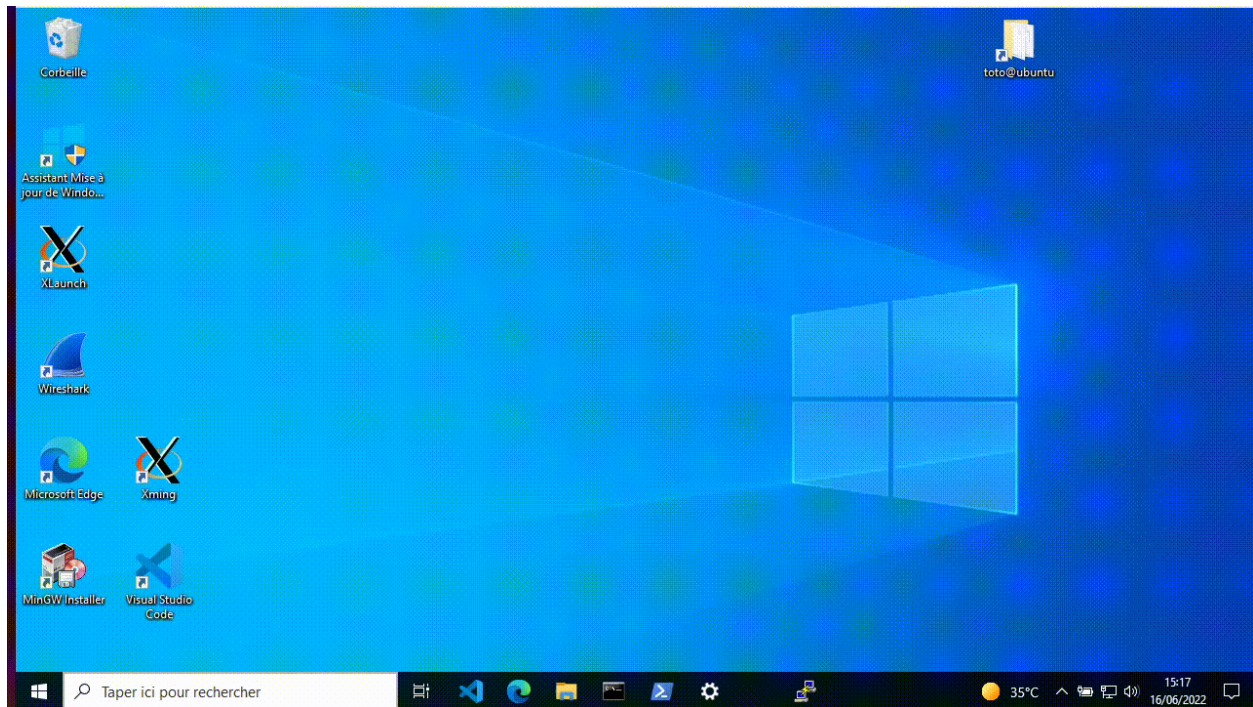


Figure 4: Lancement d'un terminal sous windows

- les périphériques (devices).

Ce SGF est simple et permet de manipuler de manière uniforme les fichiers comme les périphériques. Sous Unix, on a coutume de dire que *tout est fichier* ! Par ailleurs, le SGF d'Unix ne fait aucune supposition sur l'organisation interne des fichiers. Tout fichier est vu comme une simple suite d'octets.

Chaque fichier, quel que soit son type, dispose de droits d'accès :

- r pour *read* : droit d'ouverture et lecture du fichier
- w pour *write* : droit d'écriture et modification du fichier
- x pour *execute* : droit d'exécution du fichier, droit d'entrée pour un répertoire

Ces droits d'accès sont définis pour :

- u pour *user* : le propriétaire du fichier
- g pour *group* : les membres du groupe auquel appartient le propriétaire du fichier
- o pour *others* : les autres utilisateurs

Pour voir les propriétés d'un fichier, c'est-à-dire son propriétaire, les droits d'accès, sa taille en octets, sa date de dernière modification, etc., il faut utiliser la commande `ls -l`. Par exemple :

```
$ ls -l README.md
-rw-r--r-- 1 toto staff 249 11 jan 19:52 README.md
```

Ce fichier appartient à *toto* dans le groupe *staff*. Il fait 249 octets et a été modifié le 11 janvier à 19h52. *toto* a le droit de lire et écrire dans le fichier alors que les membres du groupe *staff* et tous les autres utilisateurs ont seulement le droit de lecture du fichier.

Le système de gestion de fichiers utilise une structure hiérarchique (arborescence) composée de répertoires et de fichiers. Chaque répertoire contient des fichiers ordinaires ou d'autres répertoires.

1.1.3.1.1 Les répertoires

Il existe un certain nombre de répertoires particuliers :

```
/    racine de l'arborescence,  
.  
..   répertoire père du répertoire courant,  
~    votre répertoire utilisateur (home directory)
```

La variable `$HOME` pointe également sur votre répertoire utilisateur, également appelé répertoire de connexion. Il s'agit en quelque sorte de votre maison, là où vous pouvez stocker vos fichiers. En général, sauf changement de votre part, il est en lecture et écriture uniquement pour vous.

Pour afficher le contenu de la variable `HOME`, dans un terminal tapez la commande : `echo $HOME`.

Nota Bene : Le symbole `$` placé devant le nom de la variable signifie que vous souhaitez accéder à la valeur de la variable.

1.1.3.1.2 Noms de fichiers

Les noms de fichiers sont limités à 256 caractères sous Unix. De préférence, n'utilisez pas d'espace mais `"_"` ou `"-"` à la place. Evitez les caractères spéciaux (`&`, `@`, `$`, `#`, ...). Le plus simple est de toujours utiliser des lettres et des chiffres.

Attention le système Unix fait la différence entre majuscules et minuscules ! Les fichiers `toto`, `Toto` et `TOTO` ont des noms différents, contrairement à Windows.

L'extension ou suffixe (optionnel) fait partie du nom, il commence par `.` et n'a pas de limite de taille (`.txt`, `.html`, `.tar.gz`, `.ps.gz`, etc.). Il permet d'indiquer le type du fichier. Il ne s'agit que d'une convention sous Unix. Aussi changer l'extension d'un fichier ne modifie pas son contenu : c'est simplement un changement de nom qui peut, cependant, modifier le comportement des programmes tenant compte des extensions.

Pour connaître le type d'un fichier, il faut utiliser la commande `file`. Par exemple :

```
file ~/.bashrc
```

1.1.3.1.3 Chemins relatif et absolu

Un chemin (*path*) est la succession de répertoires qu'il faut traverser jusqu'à l'objet que l'on veut atteindre. Ce chemin peut être absolu s'il part de la racine (notée `/`) du système de fichiers, ou encore relatif s'il part du répertoire d'où l'on effectue la commande (répertoire courant du processus exécutant la commande).

```
/
  net/
    cremi/
      A/
      B/
      quux
      C/
```

Ainsi `/net/cremi/B/quux` est un chemin absolu qui désigne le fichier `quux`. Le chemin relatif `../B/quux` désigne le fichier `quux` lorsque le répertoire courant est soit `/net/cremi/A`, soit `/net/cremi/B` ou encore `/net/cremi/C`. Notons également que le chemin `quux` désigne le fichier `quux` lorsque le répertoire courant est `/net/cremi/B`.

1.1.4 Principales commandes

Voici une liste des principales commandes à connaître pour manipuler les fichiers et les répertoires :

- `ls` (LiSt) : affiche le contenu du répertoire courant
- `cd` (Change Directory) : change de répertoire courant
- `cp` (CoPy) : copy un (ou plusieurs fichiers) vers une destination donnée

- **mv** (MoVe) : déplace un fichier vers une destination donnée
- **rm** (ReMove) : supprime un ou plusieurs fichiers
- **mkdir** (MaKe DIRectory) : crée un répertoire
- **rmdir** (ReMove DIRectory) : supprime un répertoire (vide !)
- **cat** (CATalog) : affiche le contenu d'un fichier texte
- **touch** permet de créer un fichier vide s'il n'existe pas, sinon de modifier la date de modification du fichier

man (MANual) : affiche l'aide sur une commande donnée

Pour obtenir l'aide en ligne sur une commande, il suffit d'utiliser le **man**. Pour obtenir l'aide sur une commande *cmd*, il suffit de taper :

```
man cmd
```

(en remplaçant bien sûr *cmd* par le nom de la commande voulue)

Voici par exemple le début du manuel de la commande **mkdir** :

```
$ man mkdir
MKDIR(1)          User Commands          MKDIR(1)

NAME
    mkdir - make directories

SYNOPSIS
    mkdir [OPTION]... DIRECTORY...

DESCRIPTION
    Create the DIRECTORY(ies), if they do not already exist.

    Mandatory arguments to long options are mandatory for short options too.

    -m, --mode=MODE
        set file mode (as in chmod), not a=rwx - umask

    -p, --parents
        no error if existing, make parent directories as needed
```

Faire défiler avec les flèches haut/bas, utiliser “q” pour quitter. Pour chercher plus rapidement ce qui vous intéresse, taper “/” puis le texte à chercher, puis entrée, ce qui vous amène à la première occurrence. Pour passer aux occurrences suivantes, taper “n” (comme next).

Dans l'exemple qui suit vous voyez quelques commandes tapées (ce sont les lignes qui commencent par **toto** et après le résultat de la commande. A chaque fois, on affiche le répertoire courant entre le@ et le \$.

Par exemple la première commande tapée est **pwd** qui nous affiche le chemin absolu du répertoire courant. Le résultat est **/home/toto**. Ensuite, on crée un répertoire **example** et on y entre. On crée deux sous-répertoires **dir1** et **dir2** et on y entre. On crée deux fichiers **file1.txt** et **file2.txt** dans le répertoire **dir1**. On se déplace dans le répertoire **dir1** et on crée un fichier **file3.txt** dans le répertoire **dir2**. On revient dans le répertoire **example** et on affiche l'arborescence des répertoires et fichiers. Nous terminons en compressant le répertoire **example** dans un fichier **example.zip**.

```
toto@~$ pwd
/home/toto
toto@~$ mkdir example
toto@~$ ls
example
toto@~$ cd example/
```

```

toto@~/example$ ls
toto@~/example$ mkdir dir1 dir2
toto@~/example$ ls
dir1  dir2
toto@~/example$ touch dir1/file1.txt dir1/file2.txt
toto@~/example$ cd dir1
toto@~/example/dir1$ ls
file1.txt  file2.txt
toto@~/example/dir1$ touch ../dir2/file3.txt
toto@~/example/dir1$ cd ..
toto@~/example$ cd ..
toto@~$ zip -r example.zip example/
  adding: example/ (stored 0%)
  adding: example/dir1/ (stored 0%)
  adding: example/dir1/file1.txt (stored 0%)
  adding: example/dir1/file2.txt (stored 0%)
  adding: example/dir2/ (stored 0%)
  adding: example/dir2/file3.txt (stored 0%)
toto@~$ ls
example  example.zip

```

Voici le fichier [example.zip](#) ainsi généré.

1.2 Analyse de trames avec Wireshark

1.2.1 Présentation de Wireshark

Wireshark est un outil open-source disponible sur les systèmes d'exploitation Linux/Windows/MacOS. Pour installer Wireshark, téléchargez le logiciel sur cette [page](#).

Il permet de capturer les trames qui traversent une carte réseau en temps réel, d'afficher leur contenu et d'analyser (a posteriori) ces trames c'est-à-dire d'interpréter l'en-tête de tous les protocoles encapsulés dans chacune des trames.

Pour capturer ces trames il faut avoir des droits **super-utilisateur** sur la machine. Si vous n'avez pas les droits **super-utilisateurs**, vous pouvez quand même visualiser des traces générées par wireshark ou d'autres outils comme **tcpdump**. Le format de capture de traces le plus utilisé est le format **.pcap** (pour "Packet Capture"). Par exemple le fichier [http.pcap](#) contient la trace générée suite à l'envoi d'une requête HTTP à un serveur web. Ouvrons cette trace avec wireshark :

L'interface graphique de wireshark se présente ainsi de haut en bas :

- Une barre de menu très classique.
- Une barre d'outils permettant l'accès aux principales fonctions.
- Une barre de filtre. C'est un outil puissant qui permet de filtrer les trames capturées en fonction de différents critères. Cela permet de limiter les trames affichées dans la fenêtre des trames à celles qui correspondent au filtre appliqué.
- Une fenêtre de trames qui est le cœur de l'interface de Wireshark. Elle affiche la liste des trames capturées. On peut voir dans cet exemple que la trace contient 16 trames. Nous avons également des informations détaillées pour chaque trame : l'horodatage, l'adresse source, l'adresse destination, le protocole utilisé, la longueur ainsi qu'une description synthétique de la trame. Le format d'affichage de ces données dépend du type de la trame. Par exemple, pour les champs source et destination, c'est l'adresse IP qui est affichée si elle est disponible. Si ce n'est pas le cas, cela sera l'adresse physique. Pour le protocole, wireshark affiche le protocole de plus haut niveau. Par exemple, les trames 7 à 16 utilisent toutes le protocole TCP de la couche transport. Wireshark est en mesure de déterminer que

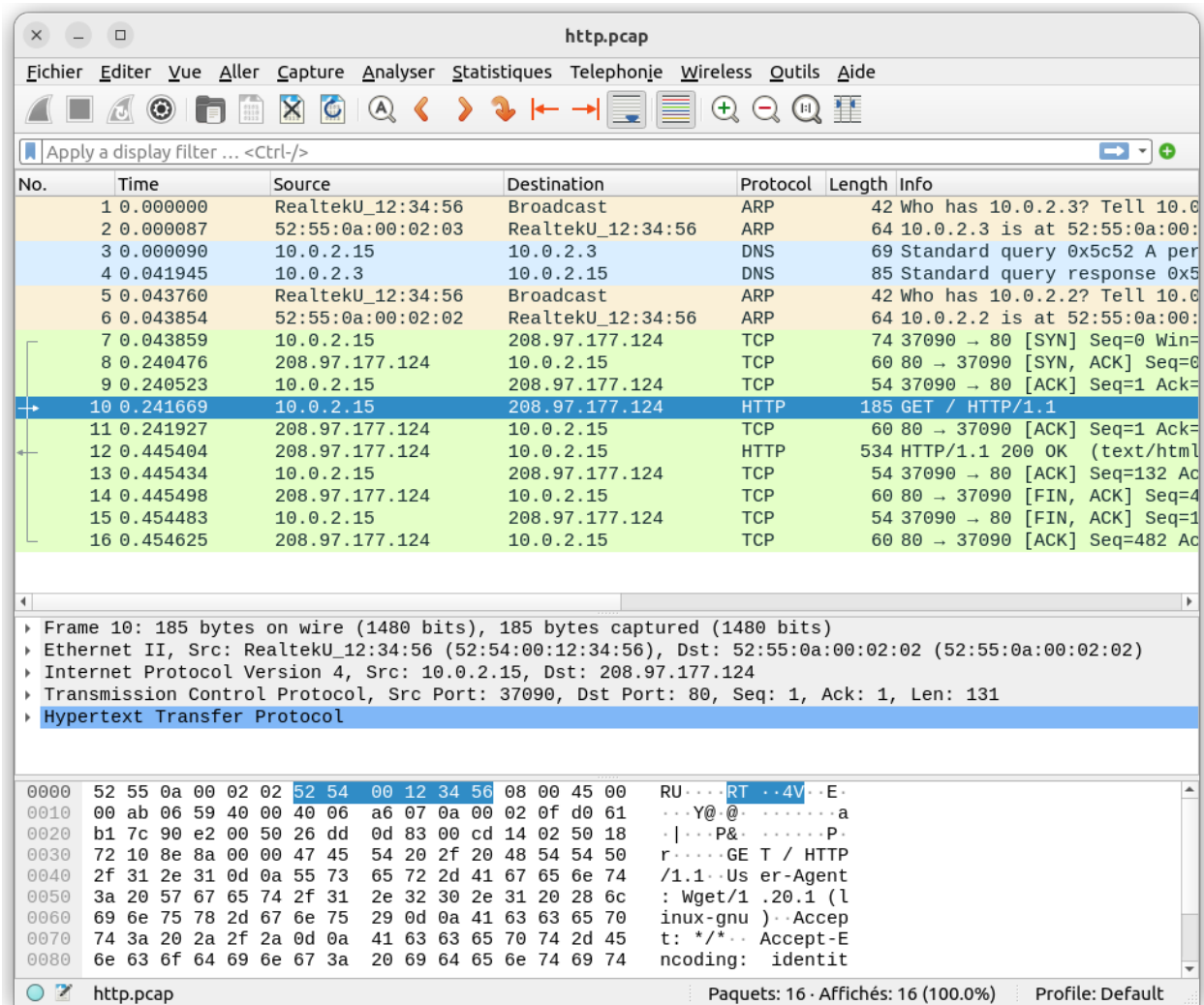


Figure 5: Ecran Wireshark

les trames 10 et 12 utilisent le protocole HTTP (au dessus de TCP), donc wireshark indique HTTP dans la colonne protocole, ce qui est plus précis d'une certaine manière. La dernière colonne contient de l'information sur la trame considérée. Là encore, wireshark fait de son mieux pour fournir un résumé synthétique du contenu de la trame.

- Fenêtre de décodage : la fenêtre de décodage est située en dessous de la fenêtre de trames et affiche les informations détaillées pour la trame sélectionnée dans la fenêtre de trames. En déroulant les différentes flèches sur la gauche, il est possible de visualiser en détails le contenu de chaque protocole encapsulé dans la trame sélectionnée. Cette fenêtre se décompose en 2 sous-fenêtre.
 - La partie du haut permet de visualiser l'encapsulation des différentes données/entêtes. Dans l'exemple ci-dessus, on voit que la trame 10 contient l'entête Ethernet (de la couche liaison) suivie de l'entête IP (couche réseau), suivie de l'entête TCP (couche transport), suivie de la partie HTTP (entête + données).
 - La partie du bas permet d'afficher le contenu de la trame octet par octet. A gauche, on retrouve le contenu au format hexadécimal et à droite la même chose mais au format ascii. Quand on sélectionne un champ dans la partie du haut, il est surligné dans la partie du bas afin de visualiser où il se situe dans la trame.

1.2.2 Inspection d'une trame

Ainsi *wireshark* permet d'autopsier chaque trame dans la partie inférieure de l'écran de wireshark. Considérons l'exemple suivant qui nous permet d'analyser la trame 10 de notre trace.

On peut déplier les différents en-têtes et voir les différents champs. De plus on peut voir comment sont codés les différents champs. La partie de gauche (en hexadécimal) est très pratique pour cela. Dans l'exemple ci-dessus, on peut voir dans quelle partie de la trame se trouve l'adresse IP destination ainsi que son codage hexadécimal. Lorsque les données contenues dans la trame sont au format text, la partie de droite est utile car elle affiche le code ascii de chaque octet. Ici, on peut voir que cette trame contient bien une requête HTTP :GET / HTTP/1.1....

1.2.3 Filtrage

Une trace contient souvent des trames provenant de plusieurs applications : c'est un peu comme si on enregistrerait dans un bar toutes les discussions qui ont lieu en même temps. Pour pouvoir analyser plus efficacement un protocole donné, nous allons avoir besoin de nous focaliser sur une partie de ces trames. Regardons comment nous pouvons faire cela grâce à la barre de filtre.

Par exemple en tapant **dns** dans la barre de filtre, on ne garde que les trames qui contiennent le protocole DNS :

On peut aussi utiliser n'importe quel champ présent dans l'entête de n'importe quel protocole connu. Par exemple pour ne récupérer que les trames qui ont pour adresse source l'adresse 208.97.177.124, il suffit de taper dans la barre de filtre :

```
ip.src == 208.97.177.124
```

Notez qu'il est également possible de combiner ces conditions avec des opérateurs logiques || (pour 'ou') et && pour 'et') :

```
(ip.src == 208.97.177.124 && http.host == "perdu.com") || dns
```

Dans ce dernier exemple, on ne garde que les messages DNS ou les requêtes/réponses HTTP émises par la machine dont l'adresse IP est 208.97.177.124 et dont l'en-tête *host* de HTTP est "perdu.com".

1.2.4 Suivre un flux TCP

Une autre fonctionnalité très utile est **Suivre un flux TCP**. Cette fonctionnalité est accessible à partir du menu contextuel (click droit) comme indiqué sur l'image suivante.

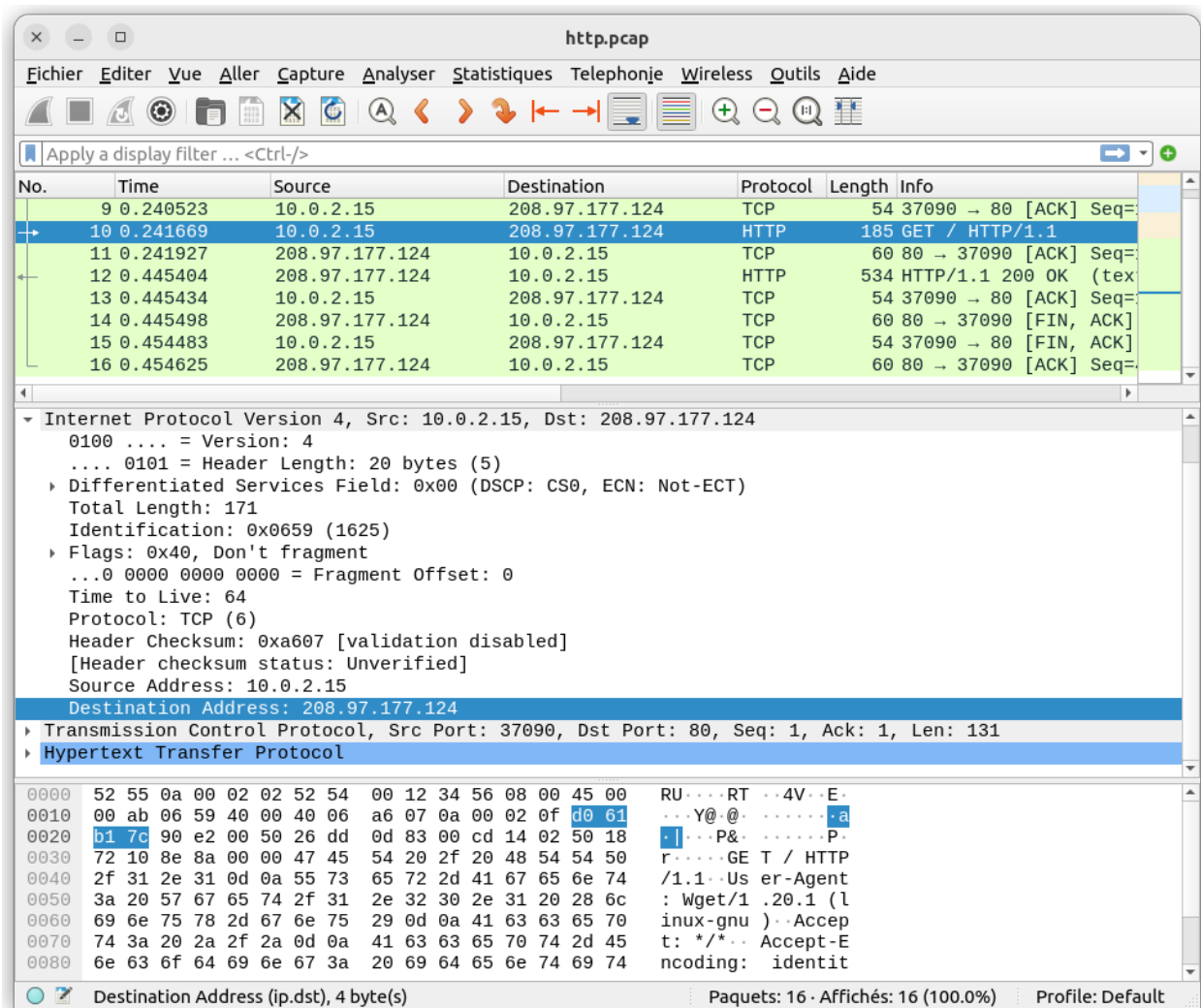


Figure 6: Wireshark : décodage d'une trame

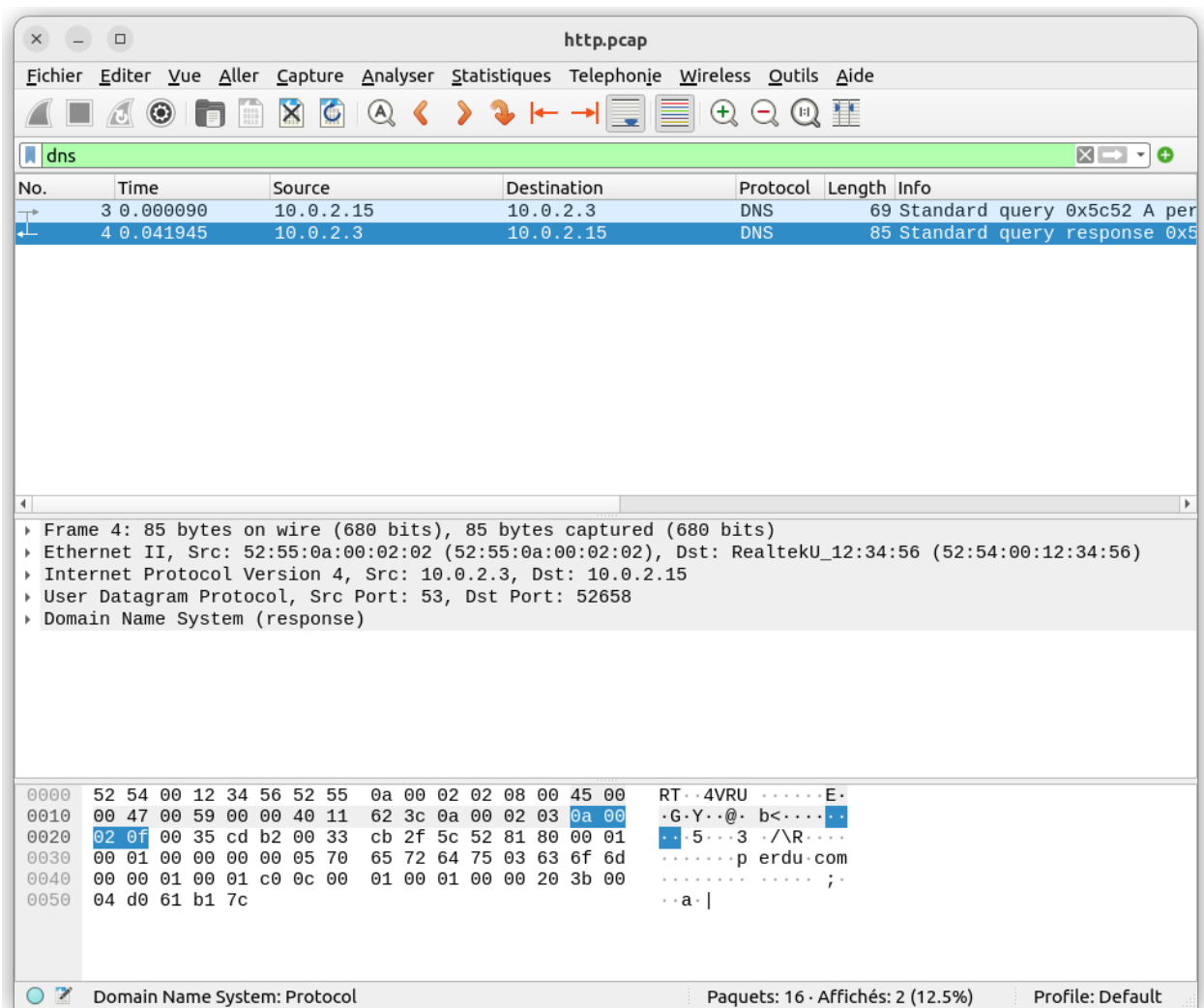


Figure 7: Wireshark : filtrage dns

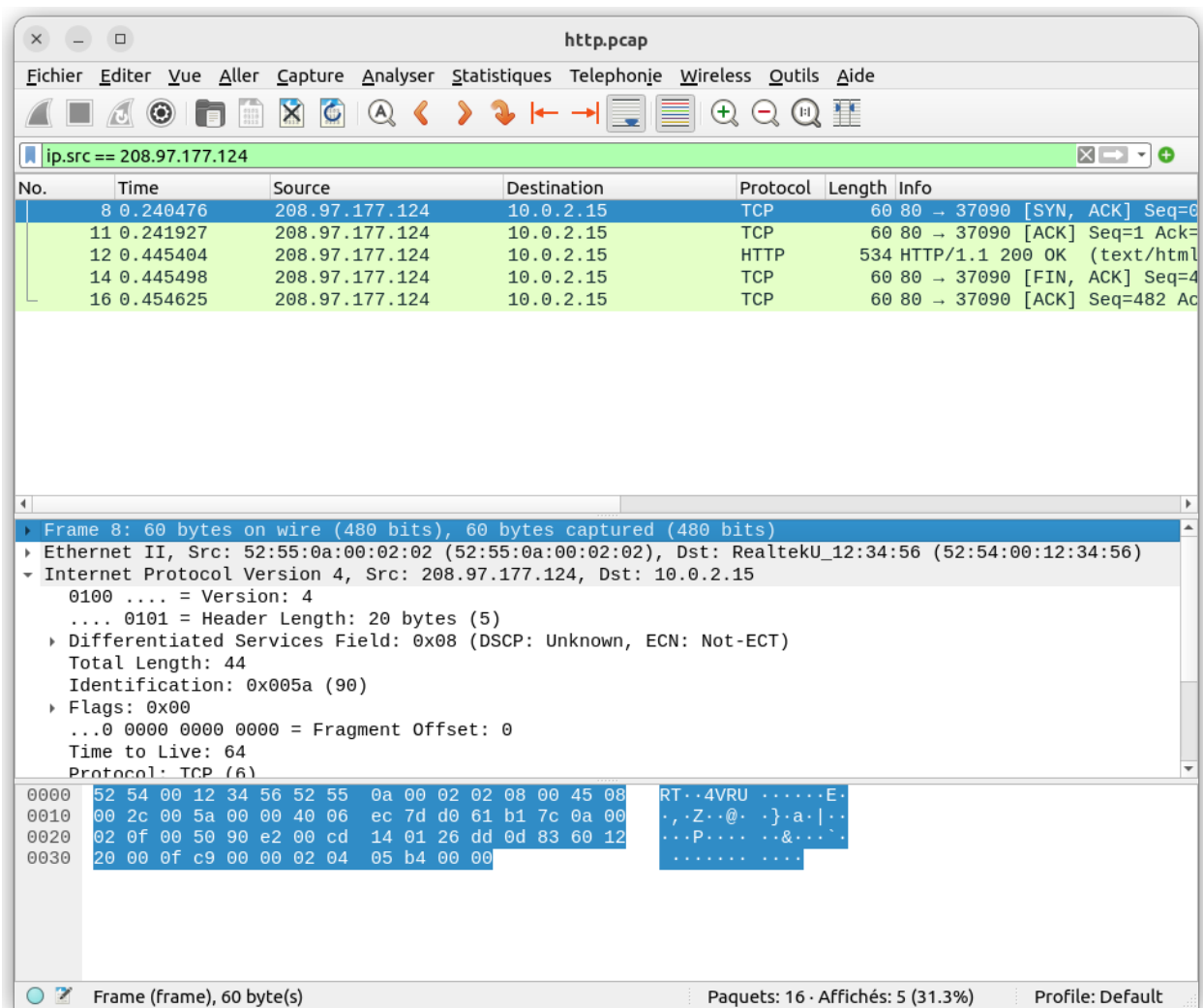


Figure 8: Wireshark : filtrage sur un champ

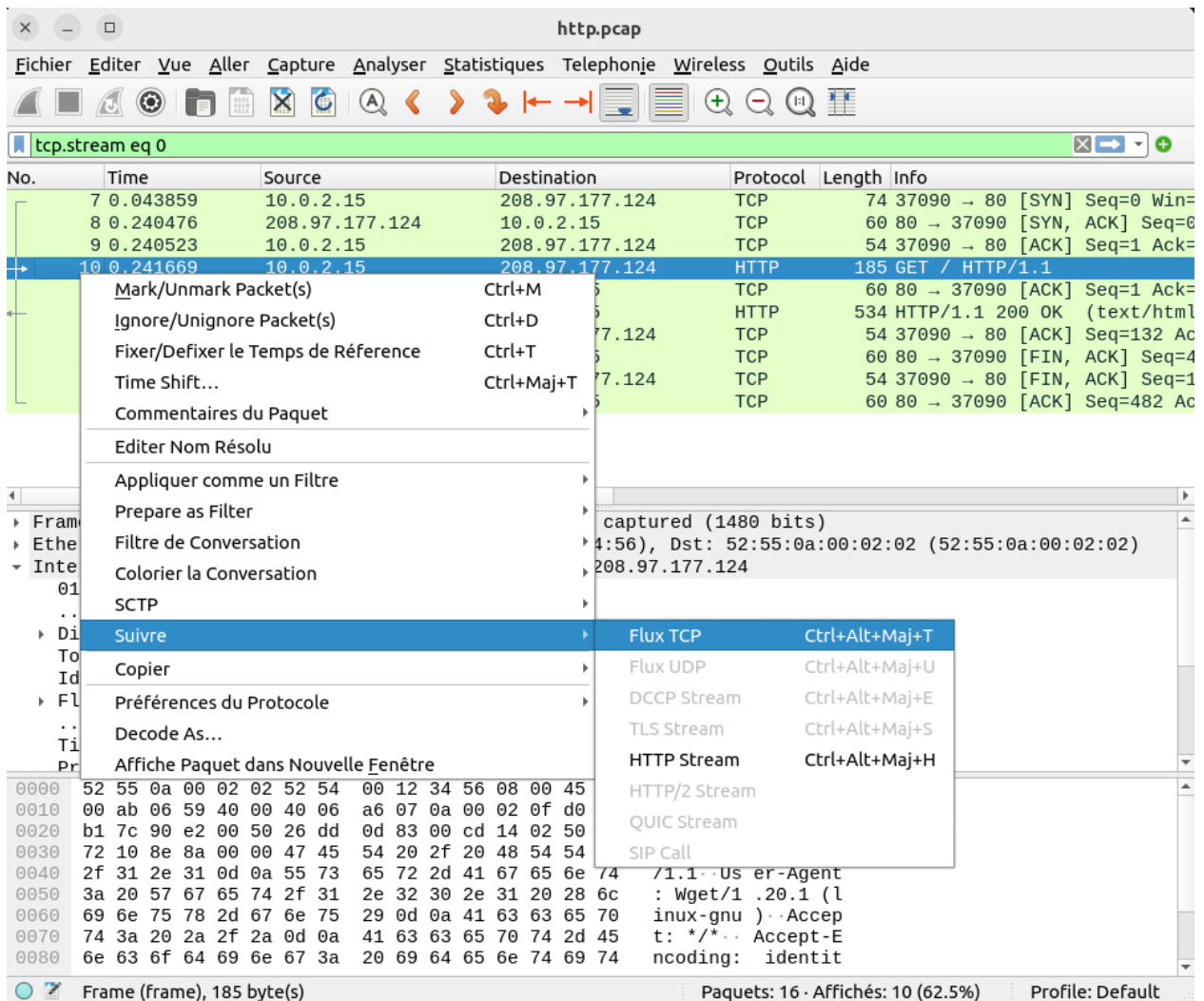


Figure 9: Wireshark : suivre flux TCP

Cette fonctionnalité permet de filtrer toutes les trames échangées dans une même connexion TCP. De plus, elle affiche le dialogue dans cette connexion sous la forme d'un texte où les messages applicatifs sortants sont affichés en rouge et les messages entrants en bleu, comme illustré dans la capture d'écran ci-dessous.

1.3 Utilisation de Filius

1.3.1 Présentation générale de Filius

Filius est un simulateur de réseaux qui permet de mettre en place une architecture avec des clients, des serveurs, des commutateurs et des routeurs. Il est possible d'intégrer des serveurs web, DNS, mail... et des clients qui font des requêtes vers ces serveurs. Il est également possible de visualiser les échanges de données entre les clients et serveurs, les tables de commutation sur les commutateurs... Il ne permet pas de configurer les machines et équipements réseaux comme vous le feriez sur du vrai matériel mais il permet néanmoins de configurer des machines et des tables de routage.

1.3.2 Installation de Filius

Vous pouvez télécharger le logiciel sur cette [page](#).

C'est en Allemand mais vous pouvez faire traduire la page par votre navigateur. Il s'agit d'une application Java qui fonctionne sous Windows, Linux et Mac.

Sous Windows, au premier lancement de Filius, il faut également installer Java si cela vous est demandé. Il est conseillé d'utiliser la version Zip plutôt que la version Windows. Il faut alors décompresser le .zip et cliquer sur `Filius.exe`.

Pour Mac, il faut récupérer le Zip, le décompresser et lancer `java -jar filius.jar` dans un terminal.

Attention de bien choisir la langue Française lors de la première ouverture de Filius. Si vous vous êtes trompés, effacez le fichier `.ini` dans le répertoire d'installation de Filius ou le dossier `~/filius` et relancez le logiciel. Sinon modifiez le fichier `.ini` pour changer la langue en enlevant le `#` devant la ligne `locale=fr_FR`.

Vous trouverez à l'adresse suivante un [Guide du Débutant](#).

Nous allons ici nous concentrer sur les fonctionnalités qui nous intéressent pour le cours.

1.3.3 Utilisation de Filius à partir d'une topologie déjà générée

L'utilisation la plus simple de Filius consiste à ouvrir une configuration existante. Prenons par exemple le fichier `premier_reseau.fls` qui contient une topologie simple avec un 2 portables, un switch, un routeur et un serveur.

Une fois ouvert, on peut voir les différents composants du réseau simulé. Pour lancer la simulation, il suffit de cliquer sur le bouton **Start** matérialisé par un triangle vert.

Une fois la simulation démarrée, on peut voir ce qu'il se passe sur chaque composant du réseau. Sur les machines, nous pouvons ouvrir l'écran et lancer des programmes comme si nous étions sur un vrai ordinateur. Par exemple sur la machine `syl` nous pouvons ouvrir un navigateur et interroger le serveur web hébergé sur la machine `opeth` en tapant l'adresse `http://192.16.0.2`.

Un peu comme avec WireShark, on peut voir les trames qui sont échangées entre les machines. Pour cela, il faut faire un click droit sur la machine et choisir **Afficher les échanges de données** :

Comme avec `wireShark` on peut voir la liste des trames échangées et pour chaque trame on peut voir le contenu de la trame présenté de manière très conviviale. Il suffit pour cela de cliquer sur la trame à inspecter et de regarder le contenu de la trame dans le bas de la fenêtre. Il est toutefois important de noter que Filius n'affiche que des informations partielles alors que `wireshark` affiche le contenu de la trame dans son intégralité.

Sur ces machines simulées, nous disposons également de terminaux qui nous permettent de taper des commandes :



Figure 10: Wireshark : suivre flux TCP

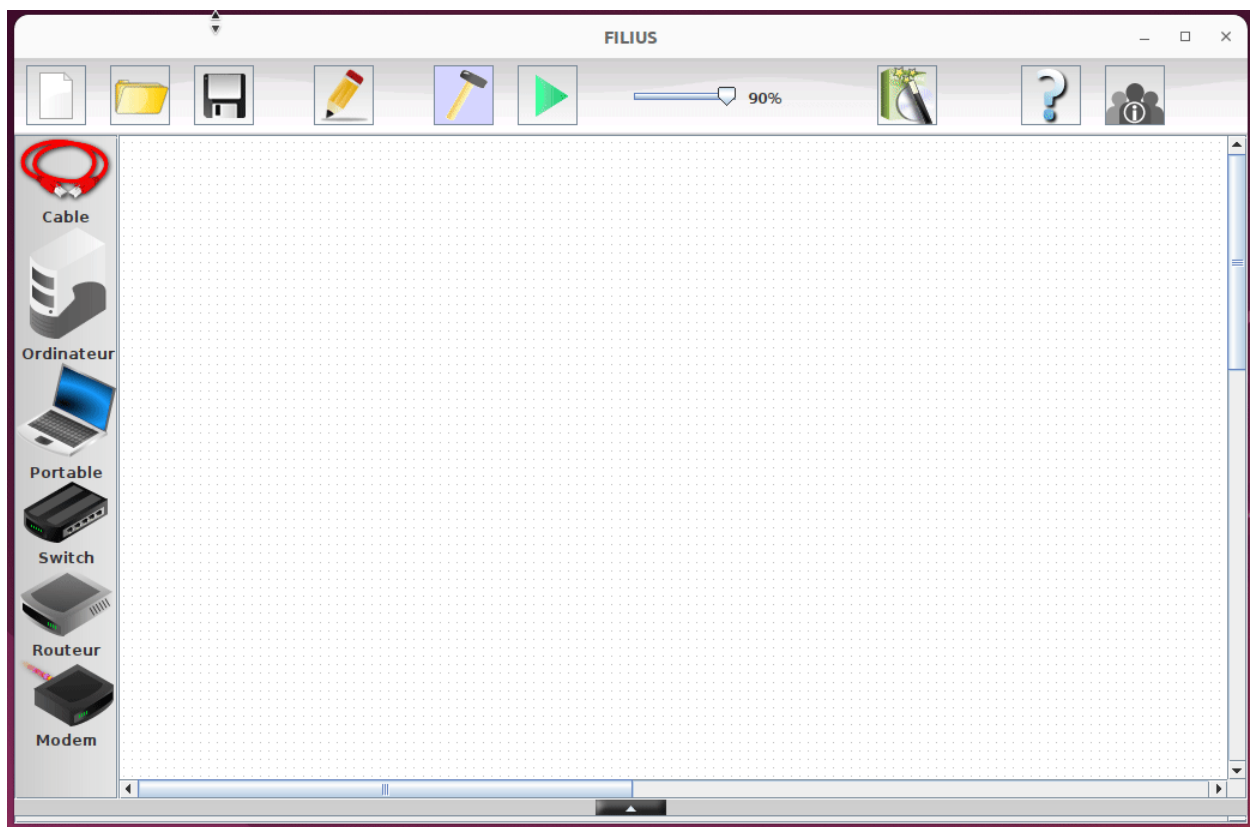


Figure 11: Ouverture d'un fichierier .fls avec Filius

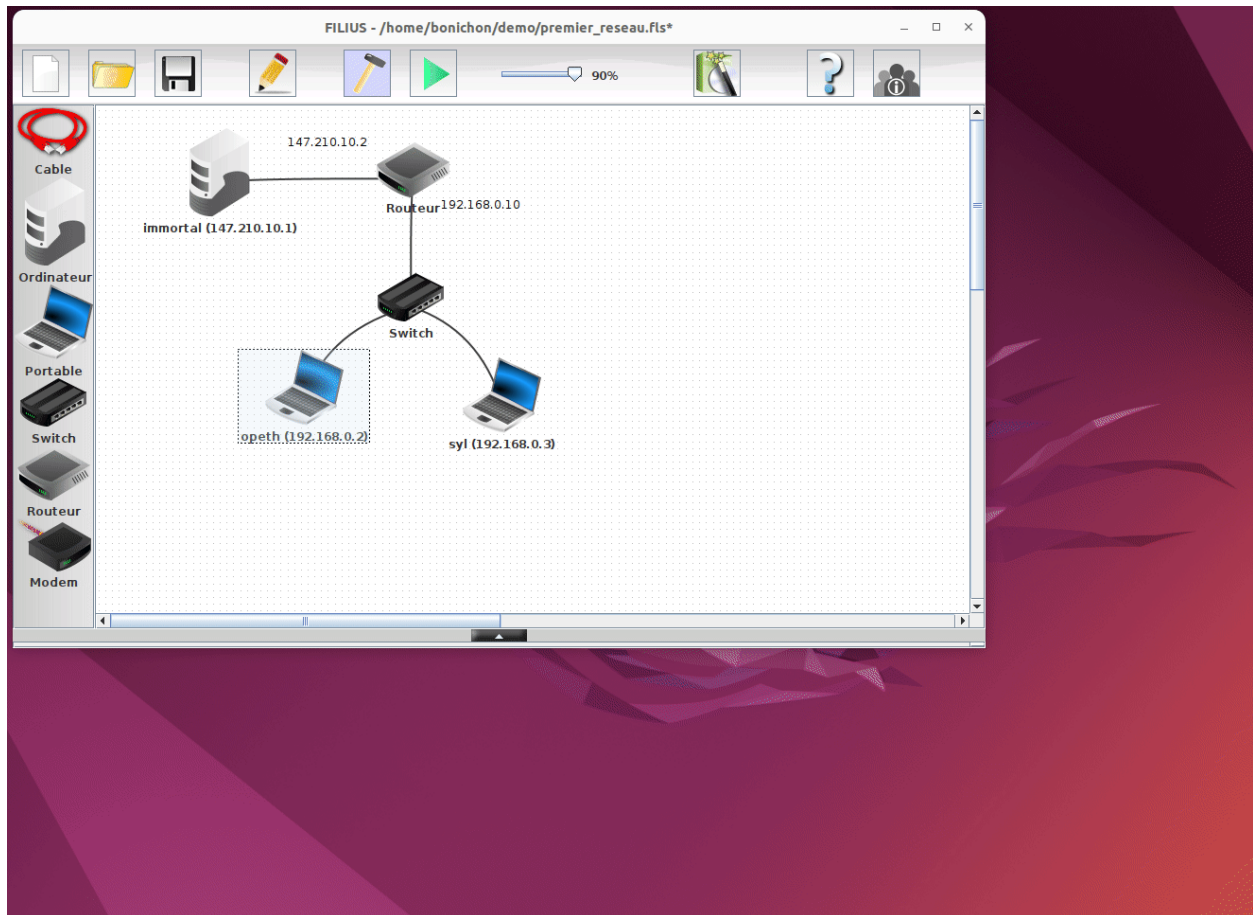


Figure 12: Lancer une simulation

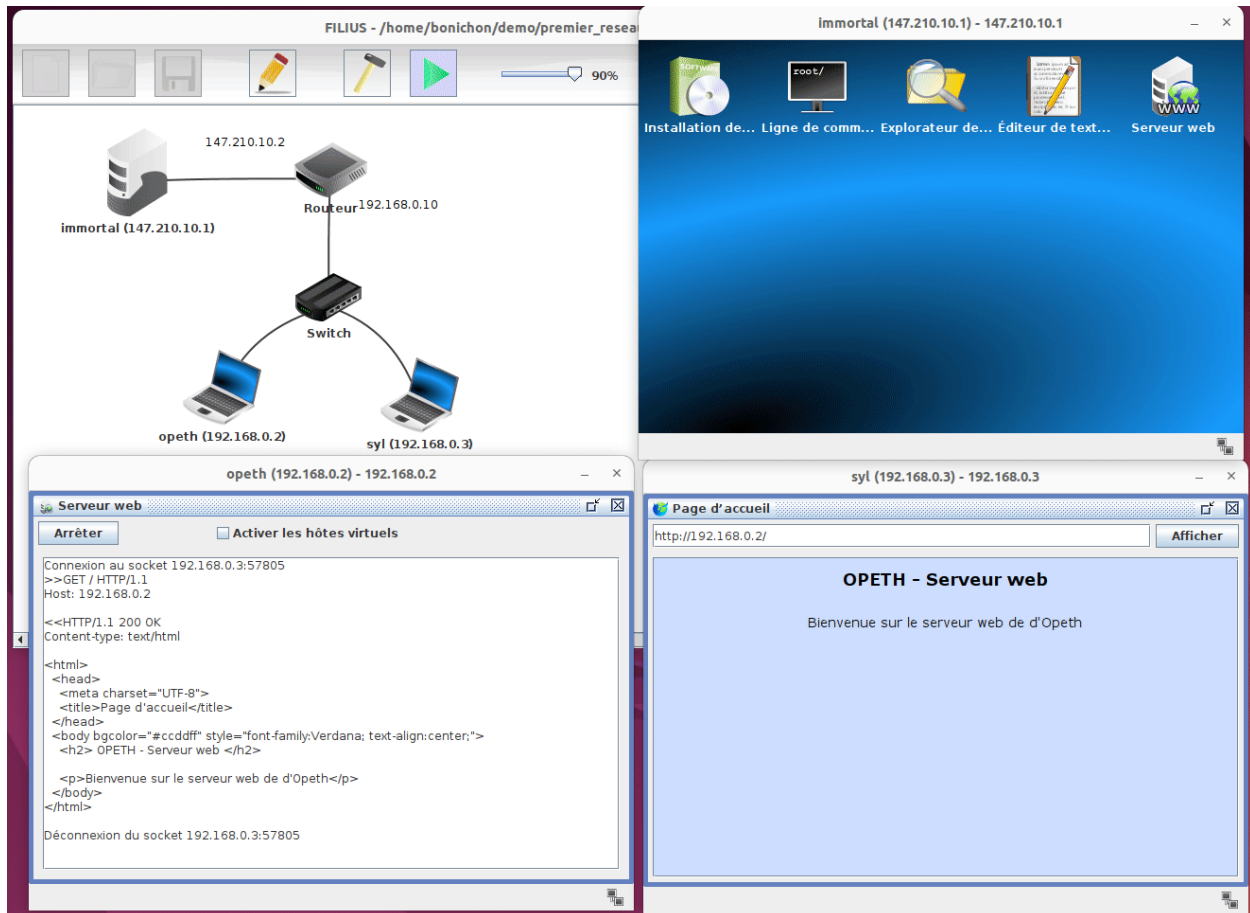


Figure 13: Inspecter les traces

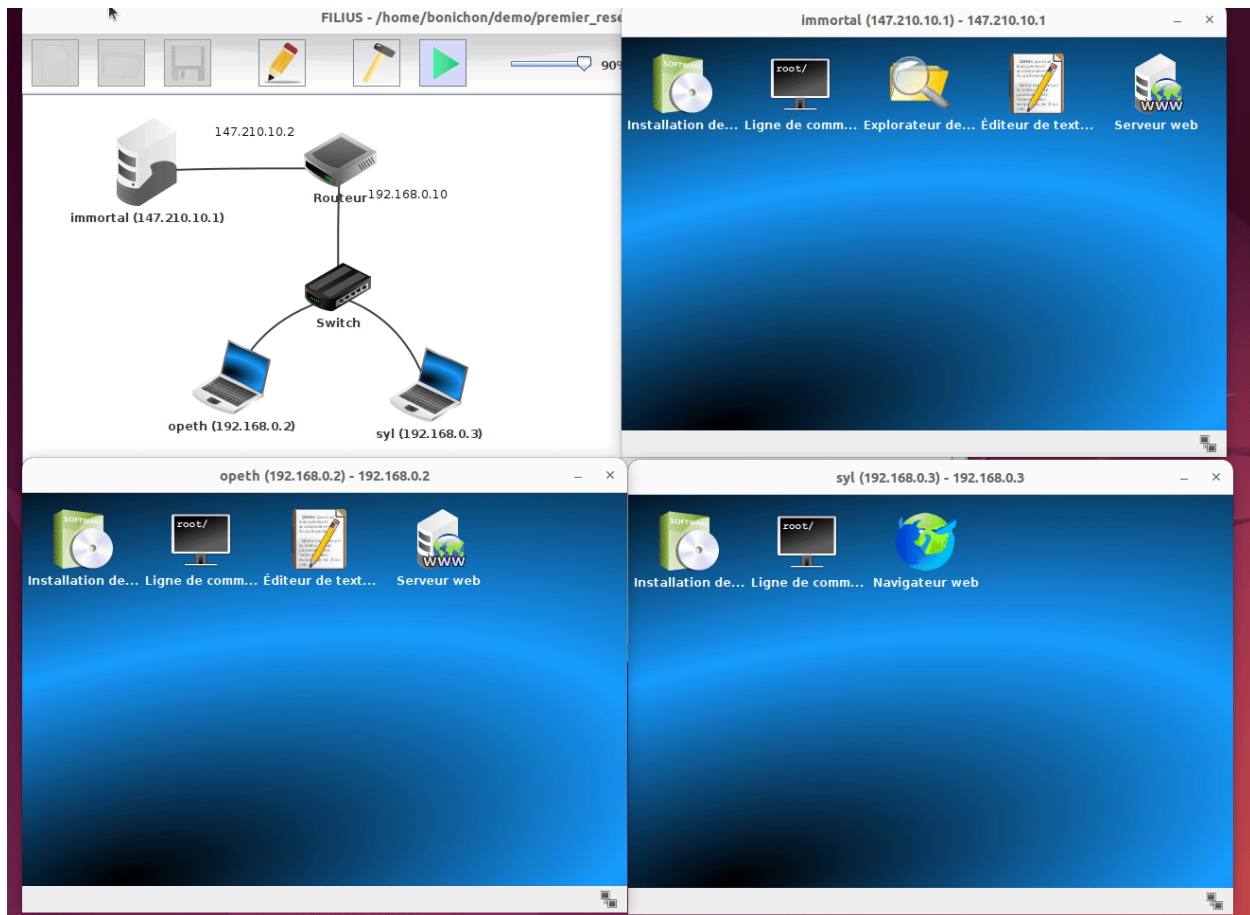


Figure 14: Utiliser le terminal

Il s'agit de terminaux très simplifiés et vous n'avez accès qu'à une liste restreinte (mais suffisante) de commande. Pour avoir la liste de ces commandes, il vous suffit de taper `help` dans le terminal.

Notez qu'il est également possible d'installer et de désinstaller des programmes sur les différentes machines. Il suffit pour cela de cliquer sur le premier programme dans l'écran de la machine :

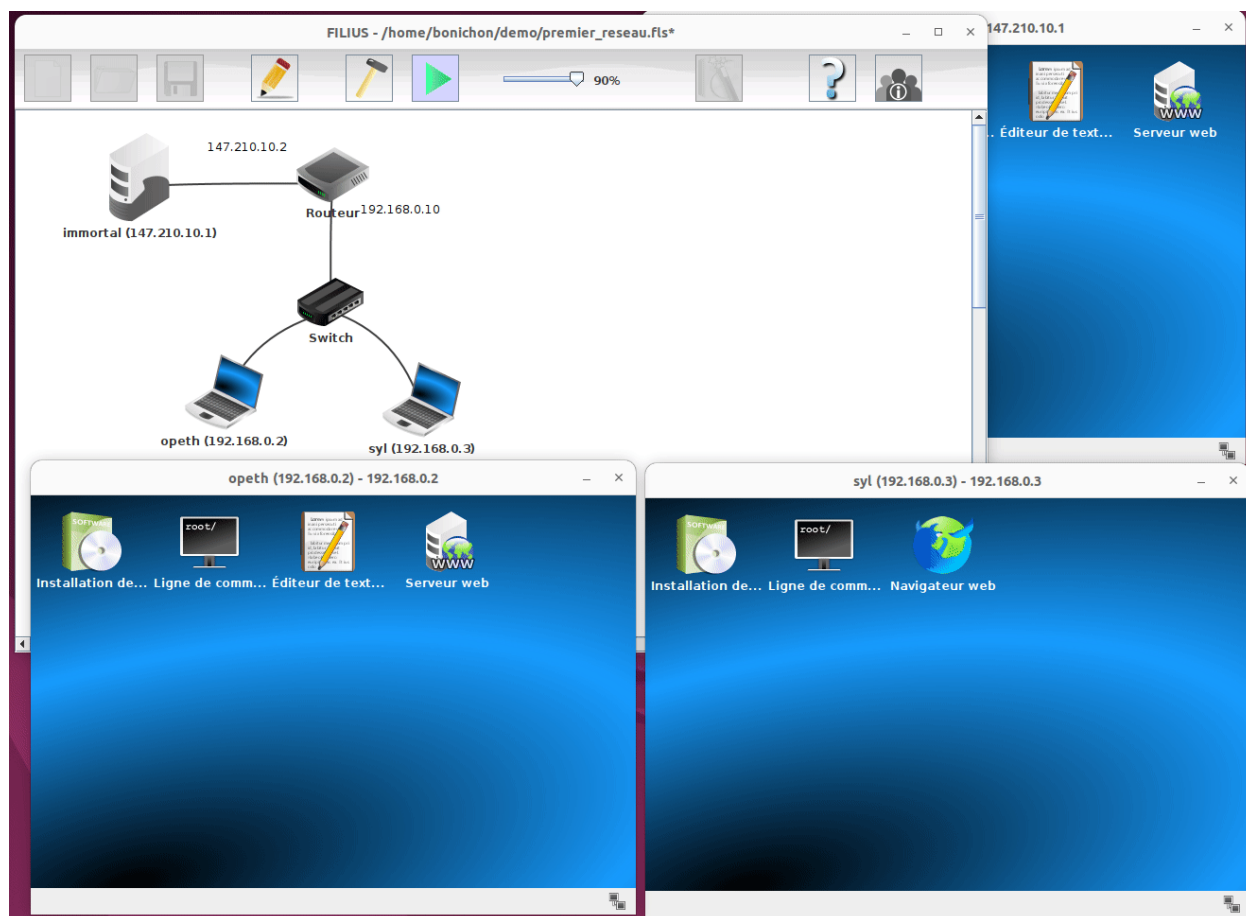


Figure 15: Installation de programme

1.3.4 Configuration des machines

Il est possible de modifier la configuration des machines. Pour cela il faut repasser dans le mode **création** en cliquant sur l'icône marteau. Une fois en mode création, vous pouvez modifier la configuration de chaque composant en cliquant dessus :

Dans notre exemple, on peut constater que le navigateur de la machine `syl` n'est pas en mesure d'accéder au serveur web hébergé sur la machine `immortal`. En regardant la configuration de la machine `syl` on peut constater qu'elle n'a pas de passerelle par défaut. En corrigeant ce problème et en relançant la simulation, on peut constater que le navigateur de la machine `syl` est maintenant en mesure d'accéder au serveur web hébergé sur la machine `immortal`.

1.3.5 Génération d'une topologie

Avec Filius, vous avez également la possibilité de construire votre propre topologie, ce que vous serez amené à faire dans le bloc 3 relatif au fonctionnement d'Internet.

Pour un petit avant goût, vous pouvez consulter cette [démonstration](#).

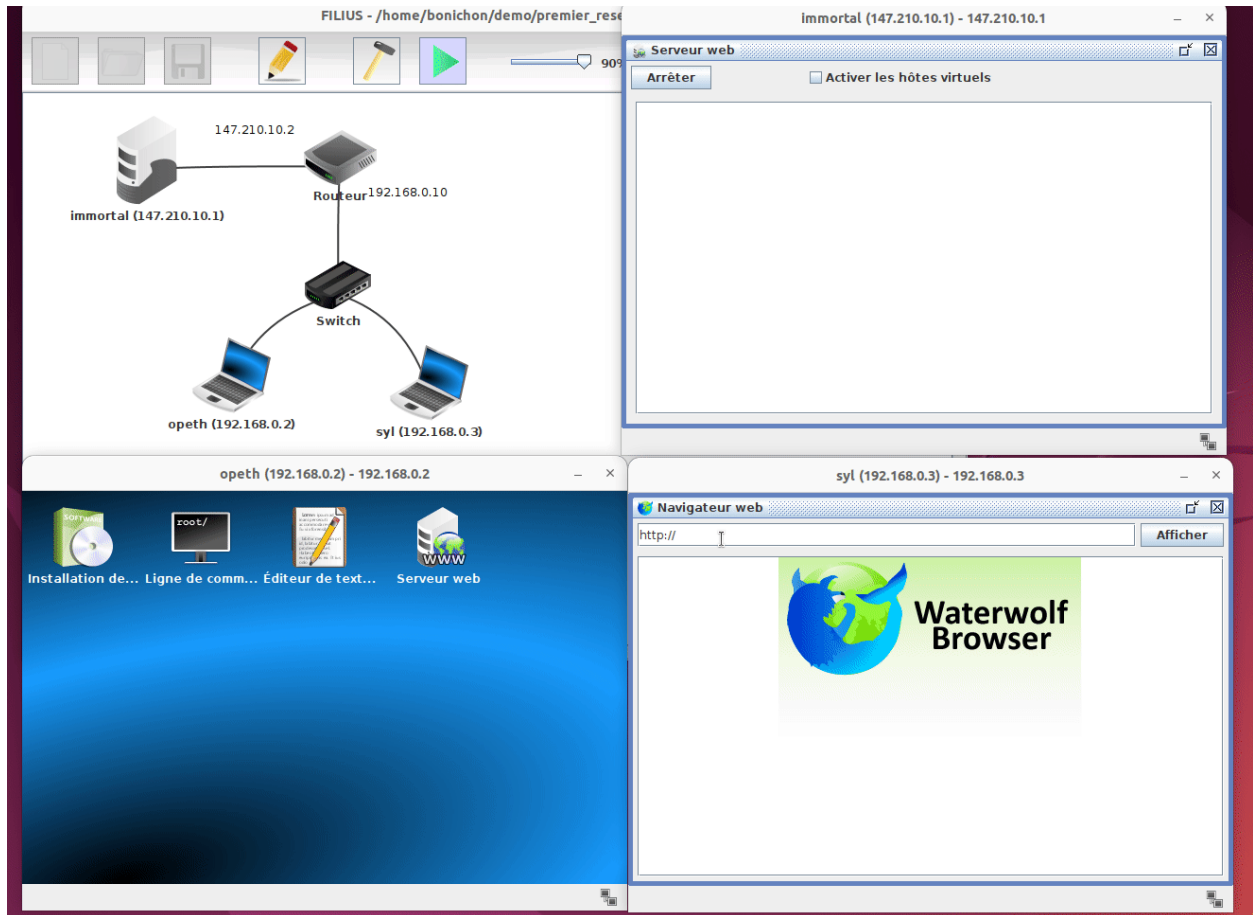


Figure 16: Modification des configurations